

Exercise 3: Sorting Customer Orders

Sorting algorithms are used to arrange data in a specific order. In this scenario, customer orders need to be sorted according to their total price so that high-value orders can be identified easily and processed first.

Bubble Sort compares adjacent elements and swaps them whenever they are in the wrong order. This process continues until the entire array becomes sorted.

Insertion Sort places each element at its correct position in the already sorted part of the array.

Quick Sort selects a pivot element and partitions the array into smaller sections. These sections are then sorted recursively.

Merge Sort divides the array into smaller parts, sorts them individually, and merges them back together to form a sorted array.

2. Implementation

```
class Order{
    int orderID;
    String customerName;
    double totalPrice;

    public Order(int orderID, String customerName, double totalPrice){
        this.orderID = orderID;
        this.customerName = customerName;
        this.totalPrice = totalPrice;
    }
}
```

Bubble Sort

```
public void bubbleSort(Order[] orders){
    int n = orders.length;
    for(int i = 0; i < n - 1; i++){
        for(int j = 0; j < n - i - 1; j++){
            if(orders[j].totalPrice > orders[j + 1].totalPrice){
                Order temp = orders[j];
                orders[j] = orders[j + 1];
                orders[j + 1] = temp;
            }
        }
    }
}
```

Quick Sort

```
public int partition(Order[] orders, int low, int high){
    double pivot = orders[high].totalPrice;
    int i = low - 1;
    for(int j = low; j < high; j++){
        if(orders[j].totalPrice < pivot){
            i++;
            Order temp = orders[i];
            orders[i] = orders[j];
            orders[j] = temp;
        }
    }
    Order temp = orders[i + 1];
    orders[i + 1] = orders[high];
    orders[high] = temp;
    return i + 1;
}

public void quickSort(Order[] orders, int low, int high){
    if(low < high){
        int mid = partition(orders, low, high);
        quickSort(orders, low, mid - 1);
        quickSort(orders, mid + 1, high);
    }
}
```

Analysis

Bubble Sort:

- Best Case: $O(n)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Quick Sort:

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n^2)$

Quick Sort is generally preferred over Bubble Sort because it performs much faster when the number of orders is large. Bubble Sort requires many comparisons and swaps, whereas Quick Sort reduces the number of operations by dividing the array into smaller sections and sorting them efficiently. Therefore, Quick Sort is more suitable for an e-commerce platform where large amounts of order data need to be processed.

Optimization

Quick Sort can be further optimized by choosing a better pivot element instead of always selecting the last element. Methods such as random pivot selection or choosing the middle element help reduce the chances of worst-case performance and improve overall efficiency.